

Class #11:

Langchain

LangChain is a framework for developing applications powered by large language models (LLMs).

Key Features

- Prompt Templates → Standardized ways to structure prompts for different use cases.
- Chains → Sequences of LLM calls and function executions.
- Memory → Enables LLMs to remember past interactions.
- Agents → Allow dynamic decision-making with LLMs to execute actions.
- Tool Integration → Enables interaction with APIs, search engines, and databases.

 Use Cases - Chatbots with memory, Advanced LLM-powered search, Question answering over custom data, Code generation and debugging.

[Join @hmaracollege for more free courses](#) 🙏

Chat Models

LLMs exposed via a chat API that processes sequences of messages as input and output a message.

 Messages - The unit of communication in chat models, used to represent model input and output.

Structure of Messages

- Role → Defines who is sending the message. Common roles:
 - system: Provides instructions on how the model should behave.
 - user: Represents the input from the user.
 - assistant: The model's response.
 - function/tool: Used in function calling (for advanced integrations).
- Content → The actual text of the message.

Prompt Templates

Component for factoring out the static parts of a model "prompt" (usually a sequence of messages). Useful for serializing, versioning, and reusing these static parts.

Runnable

The base abstraction that many LangChain components and the LangChain Expression Language are built on.

 [Join @hmaracollege for more free courses](#) 😊

 **Runnable Sequence** - A runnable sequence chains multiple steps together, where each step depends on the previous step's output.

 **Runnable Parallel** - A runnable parallel runs multiple independent tasks simultaneously and collects the results without dependencies between them.

Chains

Chains are a way to link multiple LLM calls and logic together to build more complex workflows. Instead of making a single API call to a language model, Chains allow for a structured flow of inputs, processing steps, and outputs.

Agents

Agents are systems that take a high-level task and use an LLM as a reasoning engine to decide what actions to take and execute those actions.

LangGraph

LangGraph is a low-level orchestration framework for building controllable agents. While langchain provides integrations and composable components to streamline LLM application development, the LangGraph library enables agent orchestration – offering customizable architectures, long-term memory, and human-in-the-loop to reliably handle complex tasks.

Why Use LangGraph Over LangChain

- Cycles → Particularly when dealing with cycles (looping, feedback mechanisms, or iterative reasoning), LangGraph is often the better choice.
- Faster Execution → LangGraph can run multiple steps at the same time, making it more efficient. LangChain runs steps one after another, which can slow things down.
- Smarter Decision-Making → LangGraph can dynamically choose the next step based on AI responses. LangChain needs manual coding to handle such decisions, making it harder to manage.
- Easier to Scale & Maintain → LangGraph uses a graph structure, making complex workflows organized and scalable. LangChain relies on nested chains, which get messy as things grow.

 LangSmith [Join @hmaracollege for more free courses](#) 🙄

LangSmith is a platform for building production-grade LLM applications. It allows you to closely monitor and evaluate your application, so you can ship quickly and with confidence.